



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

قسم الذكاء الصناعي واللغات الطبيعية

وظيفة الوسائط المتعددة

MultiMedia

Pixellab – Interactive Image Processing and Color Space Laboratory

عمل الطلاب

نغم خلدون مجاهد

رغد مازن الخوص

محمد نور شيخموس شيخموس

زين الدين علاء الدين رحمون

زياد راتب سليمان

2026

3	مقدمة المشروع
3	بنية المشروع والمعمارية البرمجية
4	1. تحميل الصور والتعامل معها (Open & Drag/Drop)
5	2. تحويل أنظمة الألوان
8	3. عرض القنوات والتحكم بها
9	4. تمثيل الفضاءات اللونية ثنائية وثلاثية الأبعاد
11	5. Color Quantization
17	6. عرض المعلومات الإضافية للصورة
18	7. إعادة تعيين الصورة إلى حالتها الأصلية
18	8. حفظ الصورة
18	9. مميزات عملية الحفظ
19	المصادر

يهدف مشروع PixelLab إلى بناء تطبيق مكتبي تفاعلي باستخدام لغة C# وواجهة WinForms، بحيث يعمل كمختبر بصري وتعليمي لمعالجة الصور وفهم الأنظمة اللونية المختلفة. لا يقتصر المشروع على عرض صورة أو تطبيق مجموعة عمليات سطحية عليها، بل يسعى إلى تحويل الصورة إلى مادة قابلة للفهم والتحليل، من خلال تمكين المستخدم من تحميل صورة، فحص خصائصها، تحويلها بين الفضاءات اللونية، التحكم بقنواتها، تمثيل ألوانها بصرياً في فضاءات ثنائية وثلاثية الأبعاد، اختيار ألوان منها، ومزامنة هذه الألوان مع بقية الأنظمة اللونية.

بنية المشروع والمعمارية البرمجية:

تم تنفيذ المشروع باستخدام C# WinForms، وهو اختيار مناسب لأن المشروع مطلوب كتطبيق سطح مكتب، ولأن WinForms يوفر أدوات مباشرة لبناء واجهات تفاعلية مثل الأزرار، القوائم، مربعات الصور، النوافذ المستقلة، وأحداث الماوس. ومع ذلك، لم يتم بناء المشروع بطريقة سطحية تعتمد على وضع كل الكود داخل Form1.cs، بل تم اعتماد بنية منظمة تفصل بين أجزاء المشروع المختلفة.

تم تقسيم المشروع إلى عدة طبقات ومجلدات رئيسية:

- Models
- Services
- Controls
- Forms
- Utils

يحتوي مجلد Models على الكائنات التي تمثل البيانات، مثل معلومات الصورة، إعدادات التحويل، نتائج المعالجة، بيانات القنوات، بيانات Histogram، ونماذج الألوان المختارة. أما مجلد Services فيحتوي على المنطق البرمجي والمعالجات الفعلية، مثل تحميل الصور، استخراج Metadata، التحويل بين الفضاءات اللونية، معالجة القنوات، تنفيذ Quantization، وبناء توزيعات الألوان ثنائية وثلاثية الأبعاد. بينما يحتوي مجلد Controls على عناصر واجهة مخصصة مثل لوحات عرض القنوات، لوحة اللون المختار، Canvas التوزيع اللوني، وControls أخرى. أما مجلد Forms فيضم النوافذ المستقلة مثل نافذة Quantization و 2D/3D Distribution.

يعتمد هذا التصميم على مبدأ مهم في هندسة البرمجيات وهو Separation of Concerns، أي فصل المسؤوليات. فالواجهة الرسومية لا يجب أن تحتوي على منطق معالجة الصورة، وخدمة التحويل لا يجب أن تهتم بكيفية عرض النتيجة، والنموذج لا يجب أن ينفذ عمليات واجهة. هذا الفصل يجعل المشروع أكثر وضوحاً، أسهل في الصيانة، وأسهل في التوسعة لاحقاً.

PixelLab System Architecture

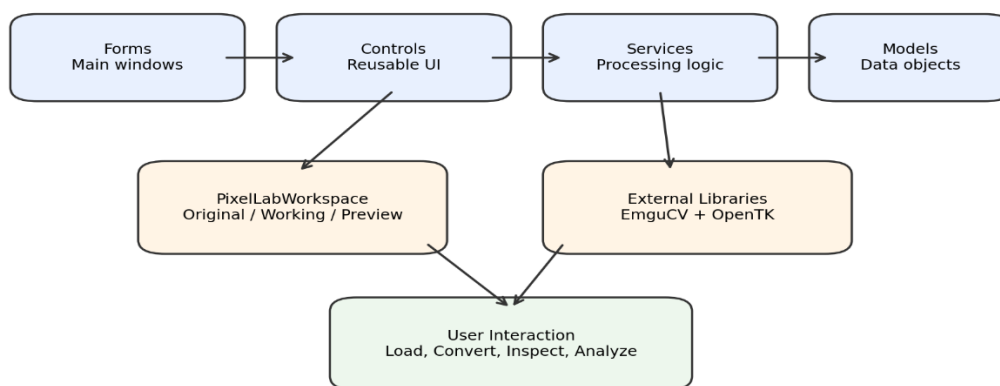


Figure 1: يوضح الشكل البنية العامة لتطبيق PixelLab وفصل المشروع إلى Forms و Controls و Services و Models.

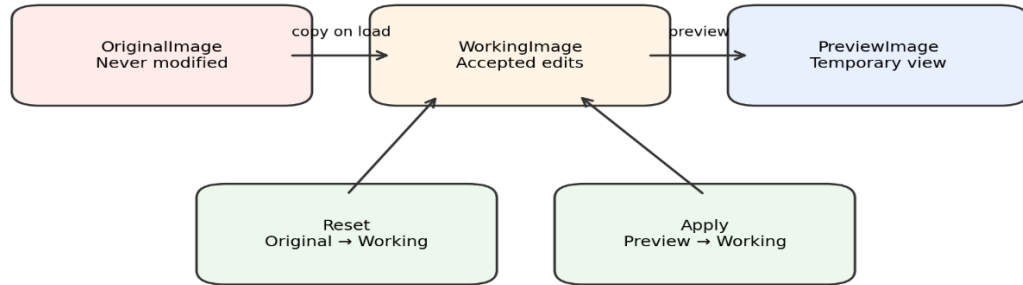
من أهم عناصر المعمارية في المشروع وجود كائن مركزي يمثل حالة العمل، وهو:
PixelLabWorkspace

ويحتوي على ثلاث صور رئيسية:

- OriginalImage
- WorkingImage
- PreviewImage

تمثل OriginalImage الصورة الأصلية كما تم تحميلها من القرص، ولا يتم تعديلها مباشرة. أما WorkingImage فهي النسخة التي تُطبق عليها العمليات المعتمدة من المستخدم، مثل Quantization أو Apply لتعديلات القنوات. أما PreviewImage فهي نسخة عرض مؤقتة تُستخدم عندما يريد المستخدم معاينة نتيجة معينة دون اعتمادها نهائياً. هذا التصميم يحقق مبدأ Non-Destructive Editing، أي التعديل غير التدميري. فالمستخدم يستطيع تجربة التحويلات والقنوات والمعاينات دون أن يفقد الصورة الأصلية، ويمكنه دائماً العودة إليها باستخدام زر Reset. هذا المبدأ مهم جداً في برامج معالجة الصور، لأنه يجعل التجربة آمنة ومرنة.

Non-Destructive Editing Model



The original image remains safe while the user experiments with previews and accepted edits.

Figure 2 : يوضح الشكل العلاقة بين OriginalImage و WorkingImage و PreviewImage ودور كل منها في منع التعديل التدميري.

تحميل الصور والتعامل معها (Open & Drag/Drop)

يهدف هذا الجزء من تمكين المستخدم من تحميل الصور داخل التطبيق بسهولة، سواء من خلال فتح ملف من الجهاز أو عبر السحب والإفلات (Drag & Drop) بعد تحميل الصورة يتم عرضها داخل واجهة البرنامج مع تهيئة الأدوات الخاصة بمعالجة القنوات اللونية.

1. فتح الصورة من الجهاز (Open File)

عند اختيار " فتح ملف " يتم تنفيذ الخطوات التالية:

1. عرض نافذة OpenFileDialog للمستخدم .
2. تحديد أنواع الملفات المسموح بها (jpg, png, bmp, gif)
3. عند اختيار صورة :
 - يتم تحميل الصورة إلى كائن Bitmap
 - تخزين الصورة داخل Workspace
 - عرض الصورة داخل PictureBox
4. تحديث رسالة الحالة باسم الصورة وحجمها

5. تفعيل لوحة التحكم الخاصة بالقنوات اللونية

6. إعادة ضبط إعدادات التعديل

2. عرض الصورة داخل البرنامج

عند عرض الصورة يتم:

1. التأكد من أن الصورة غير فارغة .
2. التخلص من أي صورة قديمة لتوفير الذاكرة (Dispose)
3. إنشاء نسخة جديدة من الصورة وعرضها .
4. ضبط طريقة العرض إلى Zoom لتناسب حجم الشاشة .
5. تحديث حالة المستخدم بمعلومات الصورة (الأبعاد).

3. السحب والإفلات (Drag & Drop)

أ. عند سحب الصورة إلى PictureBox

- التحقق من وجود ملف يتم سحبه
- التأكد أن الملف واحد فقط
- التحقق من امتداد الملف (صورة فقط)
- إذا كان صالحا: يتم السماح بعملية السحب
- إذا غير صالح: يتم رفض العملية

ب. عند افلات الصورة داخل البرنامج

- استخراج مسار الصورة
- ويتم تنفيذ نفس خطوات التحميل العادية

4. معالجة الأخطاء

تم إضافة نظام معالجة أخطاء باستخدام try/catch:

في حال فشل تحميل الصورة: يتم عرض رسالة خطأ للمستخدم بدون إيقاف البرنامج.

تحويل أنظمة الألوان

تحويل الصورة الرقمية بين عدة أنظمة لونية مختلفة وذلك بهدف تحليل الصورة ومعالجتها بطرق متعددة حسب خصائص كل نظام لوني

تم انشاء class خاص لتطبيق عمليات التحويل بين الأنظمة اللونية المختلفة وهو **ColorConversionService**

وربطها بواجهة المستخدم بحيث يمكن اختيار نظام اللون المطلوب مباشرة من قائمة منسدلة

1. آلية العمل العامة:

عند اختيار المستخدم لنظام لوني جديد من القائمة المنسدلة يتم تنفيذ الخطوات التالية:

- التحقق من وجود صورة محملة مسبقا
 - قراءة اسم النظام اللوني المختار
 - استدعاء دالة التحويل المناسبة
 - استبدال الصورة الحالية بالصورة المحولة
 - عرض الصورة الجديدة داخل الواجهة
 - تحديث رسالة الحالة لإعلام المستخدم بنجاح العملية
- تحويل نظام Rgb:** يعتبر النظام الافتراضي المستخدم في الصور الرقمية لذلك لا يتم تنفيذ تحويل فعلي وإنما يتم انشاء نسخة جديدة في الصورة الأصلية

2. تحويل الأنظمة:

❖ نظام CMY

تم تنفيذ تحويل CMY يدويا دون الاعتماد على OpenCV

مبدأ التحويل:

يعتمد نظام CMY على العلاقة العكسية مع RGB وفق المعادلات التالية:

$$Y=255-B , M=255-G , C = 255-R$$

تم اتباع الخطوات التالية:

1. التأكد من أن الصورة بصيغة 24-bit RGB
2. قفل الصورة داخل الذاكرة باستخدام LockBits
3. نسخ بيانات البكسلات إلى مصفوفة Bytes
4. المرور على جميع البكسلات.
5. تطبيق معادلات التحويل على كل قناة لونية.
6. إعادة البيانات إلى الصورة الجديدة.
7. تحرير الذاكرة باستخدام UnlockBits

سبب استخدام LockBits

تم استخدام LockBits بدل GetPixel و SetPixel بسبب:

- تحسين الأداء.
- تقليل زمن التنفيذ.
- التعامل المباشر مع الذاكرة

تم تنفيذه يدوياً أيضاً مشابه للـ CMY لكن CMYK يضيف قناة رابعة (black) K وهذا يغير طريقة الحساب والغرض من النظام المشكلة في الـ CMY عند دمج الألوان الثلاثة لا نحصل على الأسود الحقيقي بل رمادي غامق لذلك الـ CMYK أضاف الـ K لتحسين الأسود الحقيقي ولكن الـ CMYK أكثر تعقيداً لأنه:

1. يحول RGB إلى قيم بين الـ 0 و 1 (normalize) يعني يقسم على 255
2. حساب الأسود K:
يأخذ أعلى قيمة من $\max(R, G, B)$ ثم $K = 1 -$
3. ثم نحسب الـ C, M, Y وفق معادلات خاصة
4. تخزين الـ CMYK

❖ أنظمة HSV, YUV, YCbCr, LAB

في هذه الأنظمة تم استخدام مكتبة الـ Emug تعتمد عملية التحويل على الدالة :

`CvInvoke.CvtColor()`

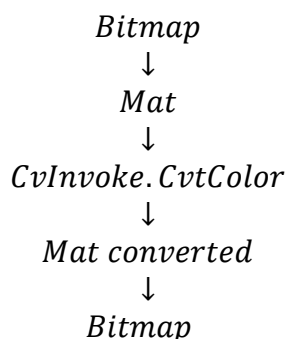
حيث يتم:

1. تحويل الصورة إلى كائن Mat.
2. تحويل RGB إلى BGR لأن OpenCV يعتمد BGR
3. تطبيق نظام الألوان المطلوب
4. تحويل النتيجة إلى Bitmap للعرض

التحويل بين الفضاءات اللونية باستخدام EmguCV:

يمثل التحويل بين الفضاءات اللونية أحد أهم أجزاء المشروع. تم استخدام مكتبة EmguCV، وهي واجهة .NET. لمكتبة OpenCV، من أجل تنفيذ التحويلات بكفاءة عالية وتجنب كتابة جميع التحويلات يدوياً. اختيار EmguCV كان قراراً تصميمياً مهماً، لأن OpenCV مكتبة معروفة ومجربة في معالجة الصور، وتوفر دوال قوية مثل `CvtColor`.

تعمل عملية التحويل وفق المسار التالي:



في WinForms يتم التعامل عادة مع Bitmap، أما EmguCV/OpenCV فيعتمد على Mat كتمثيل داخلي للصور. لذلك كان من الضروري بناء جسر بين تمثيل WinForms وتمثيل OpenCV.

ومن النقاط المهمة جدًا أن OpenCV يتعامل غالبًا مع ترتيب القنوات على أنه BGR، وليس RGB، أي أن القناة الأولى هي Blue، والثانية Green، والثالثة Red. لذلك تم استخدام تحويلات مثل:

- Bgr2Hsv
- Bgr2Lab
- Bgr2YCrCb
- Bgr2Yuv

بدل استخدام تحويلات RGB مباشرة، حتى لا يحدث انقلاب في الألوان.

يدعم المشروع الفضاءات التالية: RGB Original، Grayscale، HSV، LAB، YCbCr / YCrCb، YUV، CMYK Preview وقد تم التمييز بين طريقتين للعرض:

- Encoded View
- Round Trip Preview

تعني Encoded View عرض القيم الناتجة من الفضاء الجديد كصورة تعليمية، حتى لو لم تكن هذه القيم مخصصة للعرض المباشر على الشاشة. مثلاً عند تحويل الصورة إلى HSV، فإن القنوات تصبح H و S و V، لكن الشاشة لا تعرف معنى Hue أو Saturation، بل تتوقع RGB/BGR. لذلك قد تبدو الصورة غريبة في Encoded View، وهذا ليس خطأ، بل تمثيل بصري للقيم العددية.

أما Round Trip Preview فيقوم بتحويل الصورة إلى الفضاء المطلوب ثم إعادتها إلى BGR:

$BGR \rightarrow HSV \rightarrow BGR$

أو:

$BGR \rightarrow LAB \rightarrow BGR$

وهذا يسمح بالتحقق من صحة التحويل. فإذا عادت الصورة قريبة من الأصل، فهذا يدل على أن مسار التحويل صحيح.

وتم تنفيذ CMYK بشكل يدوي وليس عبر EmguCV، لأن OpenCV لا يوفر تحويل CMYK بنفس الطريقة القياسية المستخدمة في HSV أو LAB أو YUV. كما أن CMYK الحقيقي للطباعة يعتمد على ملفات ICC ومعايرة الطباعة والحبر والورق، لذلك تم تسميته في المشروع **CMYK Preview**، أي أنه تمثيل تعليمي وليس نظام طباعة احترافي كامل.

عرض القنوات والتحكم بها

بعد التحويل بين الفضاءات اللونية، تم تنفيذ نظام يسمح بعرض القنوات والتحكم بها. مفهوم القناة Channel يعني مكونًا واحدًا من الصورة. في RGB توجد ثلاث قنوات:

R, G, B

وفي HSV توجد:

H, S, V

وفي LAB توجد:

L, a, b

تم بناء نظام يسمح بما يلي:

- (1) عرض قناة منفردة Single Channel View
- (2) تعديل قناة عبر TrackBar
- (3) تعطيل أو تفعيل قناة

استخدم المشروع دوال مثل Split و Merge لفصل القنوات وإعادة تركيبها. عند عرض قناة منفردة، يتم تحويلها إلى صورة رمادية، حيث تعبر شدة اللون الأبيض عن قيمة عالية للقناة، واللون الأسود عن قيمة منخفضة.

من القرارات المهمة في هذا النظام أن تعطيل القنوات لا يعني دائماً وضعها على الصفر. في RGB، تعطيل قناة يعني وضعها على 0 لأن القناة تمثل كمية ضوء. لكن في LAB، القنوات a و b محايدتان حول القيمة 128 في تمثيل OpenCV ذي 8-bit، لأن القيم السالبة والموجبة يتم إزاحتها إلى المجال 0-255. وكذلك في YCbCr و YUV، قنوات اللون Cb/Cr/U/V تكون محايدة تقريباً عند 128. لذلك تم اعتماد قيم محايدة مختلفة حسب نوع القناة.

أما في HSV، فإن قناة Hue تمثل زاوية لونية، لذلك هي قناة دائرية. في OpenCV تكون قيمة H غالباً ضمن المجال 0 إلى 179. لذلك عند تعديل Hue لا ينبغي استخدام القطع Clamp فقط، بل يجب استخدام الالتفاف Wrap. فإذا تجاوزت القيمة الحد الأعلى، تعود إلى البداية، تماماً مثل دوران الزاوية على دائرة اللون. هذا النظام جعل المشروع قادراً على توضيح معنى القنوات بطريقة تفاعلية، وليس فقط نظرياً.

تمثيل الفضاءات اللونية ثنائية وثلاثية الأبعاد

1. 2D Image Color Distribution

في هذه المرحلة تم تنفيذ تمثيل ثنائي الأبعاد لتوزيع ألوان الصورة. من المهم توضيح أن الهدف لم يكن رسم الفضاء اللوني العام فقط، بل رسم توزيع ألوان الصورة المرفوعة نفسها داخل الفضاء اللوني.

أي أن التطبيق يقرأ ألوان WorkingImage، يأخذ عينة من البكسلات، يحول كل لون إلى الفضاء المطلوب، ثم يسقط مركبتين على محورين ثنائيي الأبعاد.

تم دعم عدة إسقاطات مثل:

- RGB: R-G / R-B / G-B
- HSV: H-S / H-V / S-V
- LAB: a-b / L-a / L-b
- YCbCr: Cb-Cr / Y-Cb / Y-Cr
- YUV: U-V / Y-U / Y-V
- CMYK: C-M / C-K / Y-K

كل نقطة في الرسم تمثل لوناً حقيقياً موجوداً في الصورة، وليست لوناً نظرياً. ويتم رسم النقطة بلونها الأصلي، مما يجعل التوزيع سهل القراءة بصرياً.

- تم استخدام **Sampling** لأن الصورة قد تحتوي ملايين البكسلات، ورسم كل البكسلات غير ضروري وقد يكون بطيئاً. يكفي غالباً أخذ 25,000 أو 50,000 نقطة لفهم البصمة اللونية للصورة.

من خلال هذا الرسم يمكن معرفة ما إذا كانت الصورة مشبعة أو باهتة، داكنة أو فاتحة، محدودة الألوان أو غنية، وهل تحتوي على تجمعات لونية. Clusters مثلاً، صورة سماء زرقاء ستظهر نقاطها في HSV حول Hue الأزرق، وصورة رمادية ستظهر نقاطها عند Saturation منخفضة.

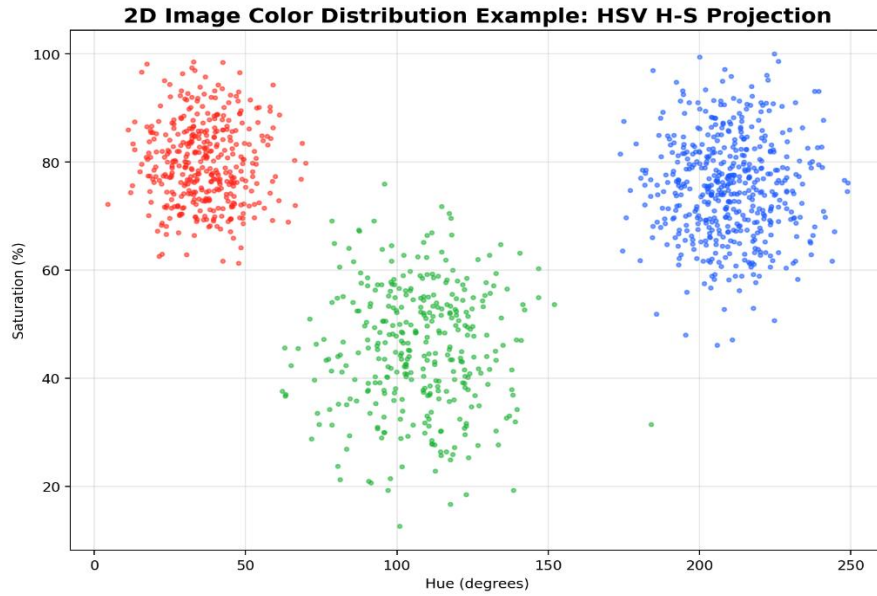


Figure 3: مثال بصري لتوزيع ألوان الصورة داخل إسقاط HSV H-S، حيث تمثل كل نقطة لوناً من الصورة.

2. 3D Image Color Distribution

تم تطوير فكرة التوزيع اللوني من D2 إلى D3 باستخدام مكتبة OpenTK / GLControl. تم وضع هذه الميزة في Form مستقل، لأن التمثيل ثلاثي الأبعاد يحتاج مساحة عرض كبيرة وتفاعلاً بالماوس.

- الفكرة هي تمثيل ألوان الصورة كسحابة نقاط ثلاثية الأبعاد:

- كل نقطة = لون مأخوذ من الصورة
- إحداثيات النقطة = قيم اللون في الفضاء المختار
- لون النقطة = اللون الأصلي للبكسل
- تم دعم عدة فضاءات:

• RGB Cube

• HSV Cylinder

• LAB Space

• YCbCr Space

• YUV Space

• CMYK C-M-K Space

- في RGB Cube يتم تمثيل اللون بالإحداثيات:

$$x = R, y = G, z = B$$

- وفي HSV Cylinder يتم استخدام Hue كزاوية، و Saturation كنصف قطر، و Value كارتفاع.

- أما في LAB فيتم استخدام:

$$x = a$$

$$y = b$$

$$z = L$$

وبالتالي سيدعم المشهد التفاعل عبر:

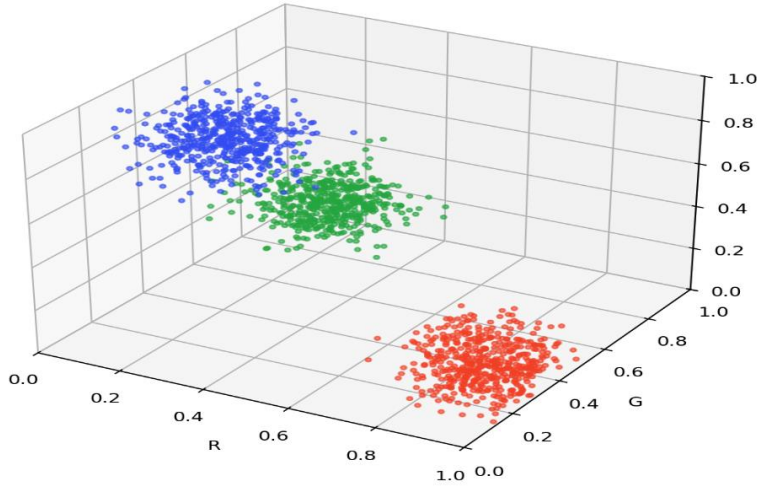
• Rotate

• Zoom

• Point Picking

يمكن للمستخدم تدوير السحابة لرؤية توزيع الألوان من زوايا مختلفة، والتكبير لفحص التفاصيل، واختيار نقطة معينة من سحابة الألوان. عند اختيار نقطة، يتم عرض قيم اللون المختار في الفضاء الحالي، ثم مزامنته مع بقية الأنظمة اللونية.

3D Image Color Distribution Example: RGB Cube



مثال لسحابة ألوان ثلاثية الأبعاد داخل RGB Cube، حيث تمثل النقاط ألوان الصورة في فضاء RGB.

Color Quantization

تم الاعتماد على عدة مصادر سيتم ذكرها لاحقاً

هي عملية تقليل عدد الألوان المختلف الموجودة في الصورة إلى عدد محدود من الصور
مثال:

لدينا صورة تحتوي على 100000 لون مختلف يمكننا تحويلها إلى:

- 256 لون
- 16 لون
- 64 لون

مع الحفاظ على الشكل العام للصورة

أهمية ل Color Quantization:

يتم استخدام هذه التقنية في العديد من التطبيقات المهمة مثل:

1. ضغط الصور: تقليل عدد الألوان يقلل حجم البيانات المطلوبة لتخزين الصورة
2. تحسين الأداء: كلما قل عدد الألوان أصبحت معالجة الصورة أسرع
3. المعالجة الحاسوبية للصور تُستخدم في:

- Image Segmentation
- Object Detection
- Pattern Recognition

كيف نمثل الألوان:

في نظام RGB يتم تمثيل كل لون باستخدام ثلاث قنوات : Red | Green | Blue كل قناة تُخزن قيمة بين 0 و 255 وبالتالي مكن تمثيل أي لون على الشكل التالي : $RGB = (R, G, B)$

في مشروعنا تم اعتبار كل Pixel نقطة داخل فضاء ثلاثي الأبعاد : $Pixel = (R, G, B)$

خوارزمية K-Means Clustering :

هي خوارزمية تجميع Clustering تُستخدم لتقسيم البيانات إلى K مجموعات مختلفة في مشروعنا:

- كل Pixel يمثل نقطة بيانات
- كل Cluster يمثل لون رئيسي
- الهدف هو إيجاد K ألوان تمثل الصورة بأفضل شكل ممكن

مبدأ عمل K-Means :

تمر الخوارزمية بعدة مراحل أساسية:

(1) اختيار عدد المجموعات K:

يقوم المستخدم بتحديد عدد الألوان التي يريد لها مثال:

- $K = 4$
- $K = 8$
- $K = 16$

كلما زادت K زادت جودة الصورة وزاد وقت التنفيذ.

(2) اختيار المراكز الابتدائية:

تسمى المراكز Centroids:

وهي تمثل الألوان الأساسية المؤقتة في مشروعنا استخدمنا (K-Means++ Initialization) بدلاً من الاختيار العشوائي التقليدي.

(3) حساب أقرب مركز لكل Pixel :

لكل Pixel يتم حساب المسافة بينه وبين جميع الـ Centroids

استخدمنا : Squared Euclidean Distance

وذلك باستخدام المعادلة:

$$D = (R1 - R2)^2 + (G1 - G2)^2 + (B1 - B2)^2$$

تم استخدام الشكل التربيعي بدون الجذر التربيعي لتحسين الأداء

(4) تجميع الـ Pixels

بعد حساب المسافات يتم إسناد كل Pixel إلى أقرب Cluster.

(5) تحديث الـ Centroids

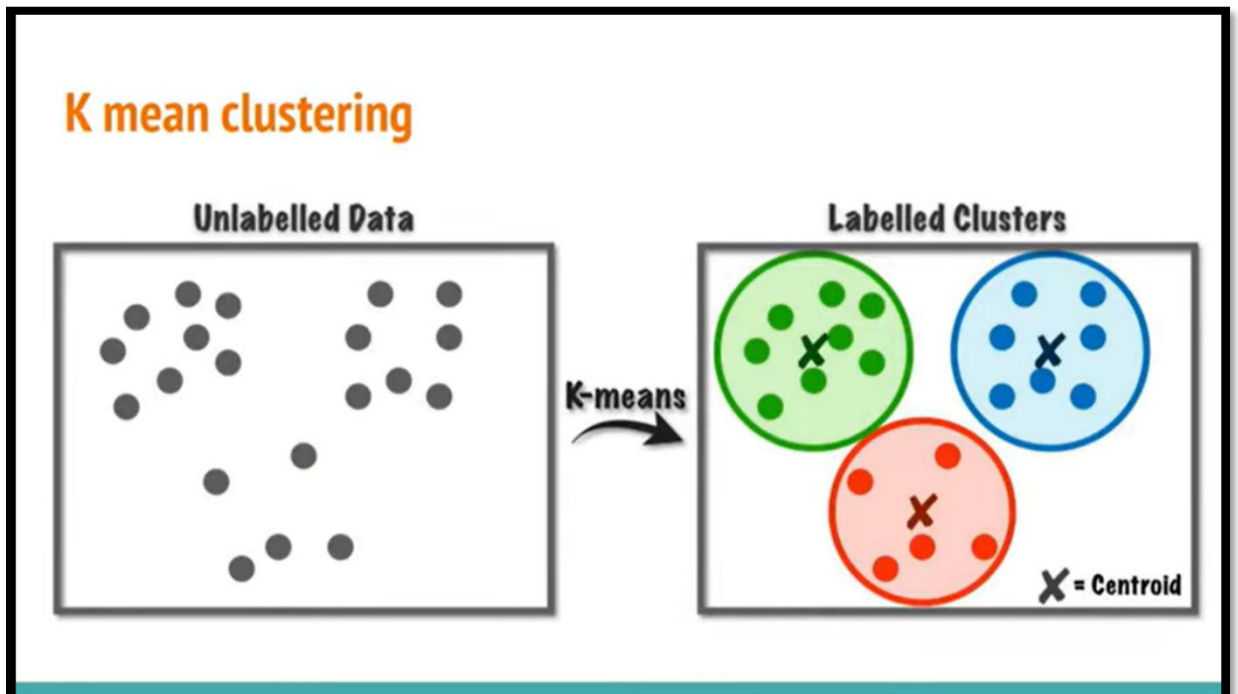
بعد انتهاء توزيع الـ Pixels يتم حساب متوسط ألوان كل Cluster

$$\text{Centroid} = (\text{Average(R)}, \text{Average(G)}, \text{Average(B)})$$

(1) التكرار

تعداد الخطوات السابقة حتى:

- تتوقف المراكز عن التغير
 - أو يتم الوصول إلى الحد الأقصى من التكرارات
- بدون ومع استخدام الخوارزمية :



التحسينات التي استخدمناها :

لم نستخدم K-Means التقليدية فقط، بل قمنا بإضافة مجموعة من التحسينات المهمة :

❖ K-Means++ :

المشكلة في K-Means التقليدية عند اختيار مراكز ابتدائية عشوائية قد يحدث:

- Convergence بطيء
- نتائج ضعيفة
- Clusters غير متوازنة

الحل

تم استخدام K-Means++ لاختيار مراكز متباعدة

الفائدة من ذلك :

- تحسين الجودة
- تسريع التقارب
- تقليل عدد التكرارات

❖ استخدام Squared Euclidean Distance :

بدلاً من استخدام: $\sqrt{x}$

تم استخدام: x فقط وذلك لأن إذا كانت:

$$A < B$$

فإن:

$$\sqrt{A} < \sqrt{B}$$

وبالتالي لا حاجة لحساب الجذر التربيعي المكلف حسابياً

الفائدة من ذلك:

- تحسين الأداء
- تقليل الزمن الحسابي

1. Stable Pixels Optimization :

تم إضافة نظام لتتبع الـ Pixels المستقرة إذا بقي الـ Pixel ضمن نفس Cluster لعدة Iterations

- يتم اعتباره Stable
- يتم إيقاف إعادة حسابه

الفائدة من ذلك :

- تقليل العمليات الحسابية
- تسريع التنفيذ

2. Adaptive Convergence :

بدلاً من الاعتماد فقط على عدد التكرارات، تم استخدام قيمة ϵ إذا أصبحت حركة الـ Centroids صغيرة جداً تتوقف الخوارزمية الفائدة من ذلك:

- تقليل عدد التكرارات غير الضرورية
- تسريع التنفيذ

3. استخدمنا LockBits :

المشكلة في GetPixel و SetPixel بطيئين جداً عند معالجة الصور الكبيرة فالحل كان باستخدام الـ Bitmap.LockBits() التي تحدثنا عنها وذلك للوصول المباشر إلى ذاكرة الصورة الفائدة من ذلك:

- زيادة كبيرة في السرعة
- تقليل زمن القراءة والكتابة

4. ايضاً استخدمنا Stride التي تحدثنا عنها:

Stride هو عدد الـ bytes المستخدم لتخزين كل سطر داخل الصورة وقد يحتوي على: Padding Bytes إضافية لمحاذاة الذاكرة استخدمنا:

$$\text{Index} = y \times \text{stride} + x \times 3$$

الفائدة من ذلك :

- قراءة صحيحة للبيانات
- منع تشويه الصور
- دعم جميع أبعاد الصور

❖ شرح عمل النظام داخل المشروع :

يمر النظام بعدة مراحل:

(a) تحميل الصورة:

يقوم المستخدم بإدخال صورة إلى التطبيق

(b) قراءة الـ Pixels:

يتم استخدام LockBits لقراءة بيانات الصورة وتحويلها إلى Byte Array

(c) تشغيل K-Means:

يتم:

- اختيار الـ Centroids
- حساب أقرب Cluster لكل Pixel
- تحديث المراكز
- تكرار العملية

(d) بناء Palette:

بعد انتهاء الخوارزمية يتم إنشاء Palette تحتوي الألوان النهائية

(e) إعادة بناء الصورة:

كل Pixel يتم استبداله بأقرب لون داخل Palette

(f) عرض النتيجة:

يعرض التطبيق:

- Preview للصورة
- إمكانية Apply أو Cancel

❖ شرح أهم التتابعات المستخدمة في هذا الطلب :

❖ التابع quantizeToolStripMenuItem_Click :

الوظيفة: يعتبر نقطة البداية لعملية Color Quantization

المهام:

- التأكد من وجود صورة
- طلب قيمة K من المستخدم
- قراءة الـ Pixels
- تشغيل K-Means
- إنشاء Preview
- عرض أزرار التحكم

❖ التابع PromptForK:

الوظيفة: عرض نافذة تسمح للمستخدم بتحديد عدد الألوان المطلوبة

المجال المسموح: من 2 حتى 256 لون.

❖ التابع RunOptimizedKMeans:

الوظيفة: تنفيذ خوارزمية K-Means مع التحسينات

أهم المهام:

- إنشاء الـ Clusters
- حساب المسافات
- تحديث الـ Centroids
- التحقق من convergence

❖ التابع InitializeKMeansPlusPlus:

الوظيفة: اختيار مراكز ابتدائية جيدة باستخدام K-Means++

❖ التابع UpdateCentroids:

الوظيفة: حساب المتوسط اللوني لكل Cluster

❖ التابع BuildLookup:

الوظيفة: إنشاء دالة تقوم بإيجاد أقرب لون داخل Palette

❖ التابع ApplyQuantizationWithLockBits:

الوظيفة: إعادة بناء الصورة النهائية باستخدام الألوان الجديدة

خوارزميات أخرى:

• Median Cut

الفكرة: تقسيم فضاء الألوان إلى صناديق متكررة

• Octree Quantization

الفكرة: بناء شجرة للألوان

• Popularity Algorithm

الفكرة: اختيار أكثر الألوان تكرار

• Wu Quantization

عرض المعلومات الإضافية للصورة

يوفر هذا الخيار للمستخدم إمكانية الاطلاع على مجموعة من المعلومات المهمة الخاصة بالصورة التي تم تحميلها داخل التطبيق. الهدف هو عرض بيانات الصورة، و مساعدة المستخدم على فهم خصائص الصور الرقمية بشكل أوضح أثناء استخدام التطبيق.

المعلومات التي يتم عرضها

يعرض النظام مجموعة من التفاصيل المهمة، وهي:

- اسم ملف الصورة.
- المسار الكامل للصورة داخل الجهاز.
- أبعاد الصورة (العرض والارتفاع).
- نسبة الأبعاد مثل 16:9 أو 4:3.
- إجمالي عدد البكسلات.
- عمق اللون (عدد البتات لكل بكسل).
- نوع أو صيغة الصورة.
- حجم الملف.

التنفيذ البرمجي

تم إنشاء خدمة مستقلة باسم ImageInfoService داخل طبقة الخدمات Services، وذلك لفصل عملية استخراج المعلومات عن واجهة المستخدم، بحيث تبقى الواجهة مسؤولة فقط عن العرض والتفاعل مع المستخدم.

طريقة عرض المعلومات للمستخدم

عند اختيار المستخدم لخيار "معلومات الصورة" من القائمة (ملف)، يتم استدعاء الخدمة السابقة ثم عرض البيانات داخل نافذة منبثقة باستخدام MessageBox.

إعادة تعيين الصورة إلى حالتها الأصلية

يوفر هذا الطلب إمكانية استرجاع الصورة الأصلية كما كانت قبل تنفيذ أي تعديل عليها. وتعتبر هذه الخاصية مهمة جدًا أثناء تجربة الفلاتر أو عمليات المعالجة المختلفة، لأنها تسمح للمستخدم بالعودة بسهولة إلى النسخة الأصلية دون الحاجة لإعادة تحميل الصورة من جديد.

يعتمد التطبيق على كائن باسم PixelLabWorkspace، وهو المسؤول عن إدارة الصور داخل النظام. تم تقسيم الصور إلى ثلاث نسخ مختلفة، لكل نسخة وظيفة محددة:

النوع	الوصف	الغرض
OriginalImage	النسخة الأصلية	الاحتفاظ بالصورة بدون أي تعديل
WorkingImage	نسخة العمل	تنفيذ التعديلات والمعالجات عليها
PreviewImage	نسخة المعاينة	عرض النتائج المؤقتة قبل التطبيق النهائي

هذا التصميم يساعد على تطبيق مفهوم Non-Destructive Editing، أي أن الصورة الأصلية تبقى محفوظة دائمًا بدون تغيير.

المميزات:

1. الحفاظ على الصورة الأصلية حتى مع تنفيذ عدد كبير من العمليات، تبقى الصورة الأصلية محفوظة دائمًا بدون أي تعديل.
2. سهولة التجربة فيمكن للمستخدم تجربة الفلاتر أو التحويلات بحرية، ثم العودة للوضع الأصلي بضغطة زر واحدة.
3. تنظيم أفضل للكود وتقسيم الصور إلى نسخ مختلفة جعل إدارة العمليات أكثر وضوحًا وأسهل في التطوير لاحقًا.

حفظ الصورة

يتيح هذا الجزء للمستخدم حفظ الصورة الحالية بعد الانتهاء من التعديلات، مع إمكانية اختيار مكان الحفظ والصيغة المناسبة.

يدعم التطبيق أكثر من نوع للصور مثل:

- JPG
- PNG
- BMP

مميزات عملية الحفظ

1. دعم أكثر من صيغة للصور.
2. إمكانية التحكم بجودة صور JPEG.
3. معالجة الأخطاء باستخدام try/catch.
4. عرض رسائل واضحة للمستخدم عند نجاح أو فشل العملية.
5. تحديث شريط الحالة داخل التطبيق بعد الحفظ.

- Wikipedia - Color Quantization :

- <https://youtu.be/4b5d3muPQmA?si=IsDP9QxCjoZENl8Y>
- Celebi, M. E. (2023). "Forty Years of Color Quantization: A Modern, Algorithmic Survey". *Artificial Intelligence Review*. **56** (12): 13953–14034. doi:[10.1007/s10462-023-10406-6](https://doi.org/10.1007/s10462-023-10406-6).
- Celebi, M. E. (2011). "Improving the performance of k-means for color quantization". *Image and Vision Computing*. **29** (4): 260–271. arXiv:[1101.0395](https://arxiv.org/abs/1101.0395). Bibcode:2011arXiv1101.0395E. doi:[10.1016/j.imavis.2010.10.002](https://doi.org/10.1016/j.imavis.2010.10.002). S2CID [9557537](https://doi.org/10.1016/j.imavis.2010.10.002).
- "[NeuQuant: Neural Image Quantization](#)". Archived from [the original](#) on 2006-06-14. Retrieved 2006-05-02.
- Bah, Tavmjong (2007-07-23). "[Inkscape » Tracing Bitmaps » Multiple Scans](#)". Retrieved 2008-02-23.
-

- MathWorld - Euclidean Distance :

- <https://mathworld.wolfram.com/EuclideanDistance.html>

- IBM - K-Means Clustering:

- <https://www.ibm.com/think/topics/k-means-clustering>

- ❖ K-Means++:

- [kMeansPP-soda.pdf](#)